

INFORME TÉCNICO DE AUDITORÍA Y MODERNIZACIÓN DE SOFTWARE LEGACY

Asignatura: Ingeniería de Software

Guía de Laboratorio #3: Migración de sistemas bancarios de C++ a Java 21 con asistencia socrática de IA

El presente informe documenta el proceso de auditoría de código, análisis de memoria y refactorización arquitectónica de un módulo bancario heredado (*legacy*) escrito en C++ hacia una plataforma moderna y administrada en Java 21, cuyo objetivo principal consistió en eliminar fallas críticas de seguridad de memoria (fugas de memoria y falta de encapsulamiento) e implementar un diseño orientado a objetos robusto utilizando herencia, abstracción, polimorfismo y gestión determinista de recursos mediante la interfaz `AutoCloseable`.

AUDITORÍA DE MEMORIA Y ANÁLISIS C++ LEGACY (FASE 1)

1. Fugas de Memoria (*Memory Leaks*) Identificadas

Durante la auditoría del archivo *SistemaBancarioLegacy.cpp*, se identificó una severa falla de gestión de memoria dinámica en el montón (*Heap*):

- **Asignación sin liberación:** En la función *crearCuenta*, se realiza la reserva explícita de memoria dinámica mediante los operadores `new CuentaLegacy()` y `new char[50]` para el arreglo del atributo titular.
- **Comportamiento del puntero titular:** Debido a que el programa no implementa la instrucción `delete[] c->titular;` ni `delete c;` antes de que los punteros pierdan su alcance (*scope*), el bloque de memoria asignado al arreglo de caracteres queda en estado "huérfano" en el *Heap*. La dirección de memoria asignada deja de ser accesible, pero el sistema operativo mantiene ese espacio como ocupado, entonces en un entorno transaccional con millones de registros, esto causa un agotamiento progresivo de la memoria RAM del servidor.

2. Debilidades Arquitectónicas y de Encapsulamiento

- **Uso de struct sin Encapsulamiento:** En C++, los miembros de una estructura son públicos por defecto, los atributos *saldo* y *limiteSobregiro* están expuestos sin mecanismos de protección, por lo que cualquier función o componente externo puede modificar directamente el saldo violando la integridad de los datos financieros.
- **Control mediante lógica procedimental:** La distinción entre productos financieros (Cuenta de Ahorros vs. Cuenta Corriente) se maneja mediante estructuras condicionales simples, lo cual impide la extensibilidad del sistema frente a la incorporación de nuevos productos financieros.

REDISEÑO Y ARQUITECTURA ORIENTADA A OBJETOS EN JAVA 21 (FASE 2)

1. Abstracción y Encapsulamiento

- **Clase Abstracta CuentaBancaria:** Representa la entidad financiera base. Es declarada como *abstract* para evitar que sea instanciada directamente, garantizando que todo objeto pertenezca a un producto bancario concreto.
- **Ocultamiento de datos:** Los atributos *numeroCuenta*, titular y saldo son declarados como *private*. Se proveen métodos públicos getters para la lectura controlada y un método protegido *protected void setSaldo(double nuevoSaldo)* para permitir que únicamente las subclases actualicen el saldo bajo reglas de negocio estrictas.

2. Herencia y Polimorfismo

- **Subclase CuentaAhorros:** Hereda de *CuentaBancaria* e implementa la regla de negocio que prohíbe sobregiros. Define la implementación del método abstracto *aplicarComisionMensual()*, calculando los rendimientos por tasa de interés fija y descontando la comisión de manejo mensual.
- **Subclase CuentaCorriente:** Hereda de *CuentaBancaria* y añade los atributos *cupoSobregiro* y *comisionSobregiro*. Sobrescribe el método *retirar()* para permitir saldos negativos hasta el límite del cupo autorizado. En *aplicarComisionMensual()*, evalúa si la cuenta presenta saldo negativo para calcular el cobro por interés de mora diario/mensual correspondiente.
- **Polimorfismo:** El método abstracto *public abstract void aplicarComisionMensual()* obliga a cada subclase a definir sus reglas de negocio particulares. El sistema puede procesar colecciones genéricas de tipo *CuentaBancaria* invocando *aplicarComisionMensual()* de forma polimórfica sin necesidad de condicionales *if/else*.

GESTIÓN DE RECURSOS CON AutoCloseable Y try-with-resources (FASE 3)

1. Cierre Determinista vs. Garbage Collector

En Java, el *Garbage Collector* (GC) gestiona la liberación de memoria en el Heap de forma no determinista, por lo que el desarrollador no puede predecir el momento exacto en que el GC destruirá un objeto, confiar en el GC para liberar recursos externos del sistema operativo genera fugas de descriptores de archivos.

2. Implementación de AutoCloseable y Equivalencia con RAII

- Se implementó la clase *RegistroAuditoriaBancaria* implementando la interfaz *java.lang.AutoCloseable* y sobrescribiendo el método *close()*.
- **Uso de try-with-resources:** Al instanciar *RegistroAuditoriaBancaria* dentro del bloque *try (...)*, Java garantiza que al salir del bloque (ya sea por ejecución normal o por una excepción), el método *close()* se ejecute de forma inmediata e incondicional.
- **Equivalencia RAII:** Este mecanismo emula de manera idéntica el patrón C++ **RAII** (*Resource Acquisition Is Initialization*), donde el destructor de un objeto libera automáticamente los recursos cuando el objeto sale de su ámbito de ejecución (*scope*).

ENTREGABLES Y EVIDENCIAS DE IA SOCRÁTICA

Repositorio Git: Código fuente C++ auditado, código Java refactorizado e historial de commits.

Link: [RepositorioLab3](#)

Configuración de Gema IA: Acceso a las instrucciones y prompt socrático configurado en Gemini.

Link: [Gem de Gemini](#)

Historial Chat Socrático: Registro público del diálogo interactivo sobre las Fases 1, 2 y 3

Link: [Conversación IA Socrática](#)

CONCLUSIONES

La auditoría del componente C++ permitió constatar que la gestión explícita de memoria mediante new sin la contraparte delete conduce indefectiblemente a fugas de memoria en el Heap.

La migración a Java 21 resolvió las vulnerabilidades de seguridad de datos mediante el principio de encapsulamiento y permitió estructurar una jerarquía financiera modular sustentada en herencia y polimorfismo.

La implementación de AutoCloseable mediante la sintaxis try-with-resources demostró ser la alternativa óptima para garantizar el cierre determinista de recursos críticos de auditoría, equiparándose a la seguridad que ofrece el patrón RAII en C++.